

INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO

Estrategia de Articulación EnContexto · Departamento de Sistemas Digitales

SolarBI Pascual: Power BI y Grafana, gobernanza de datos, automatización ETL e IoT

Integrantes

Daniela Montoya Quintero

Isaac Madera Jiménez

Curso: Inteligencia de Negocios · **Grupo:** 50

Semestre: 2026-II

Docente: Ramiro Grisales Montoya

Fecha: 6 de octubre de 2026

Parte A · Consulta

Bloque 1 · Power BI y Grafana

D1. ¿Qué es Grafana?

Grafana es una plataforma de visualización y monitoreo que consulta los datos donde están, sin copiarlos [1]. Sus componentes: los *data sources* conectan con la fuente (en SolarBI, PostgreSQL); un *panel* dibuja el resultado de una consulta; un *dashboard* agrupa paneles y se guarda como JSON; las *variables* parametrizan las consultas; el *alerting* evalúa reglas cada cierto tiempo y avisa por canales como correo o Slack [1]. Grafana OSS es gratuito, de código abierto y se instala y mantiene en servidor propio, como hace nuestro `docker-compose.yml`1. Grafana Cloud es el mismo producto alojado y operado por Grafana Labs, con plan gratuito limitado y planes de pago [2].

D2. Power BI vs. Grafana

Criterio	Power BI	Grafana
Propósito principal	Inteligencia de negocios y análisis de indicadores [3]	Monitoreo y observabilidad de sistemas y equipos [1]
Usuario típico	Analista de negocio y directivo	Operador, ingeniero de planta o de operaciones
Conexión a los datos	Conectores; Import, DirectQuery o conexión en vivo [4]	<i>Data sources</i> que consultan en cada carga del panel [1]
Modelo semántico y lenguaje	Modelo con tablas, relaciones y medidas en DAX	Sin modelo propio: cada panel lleva su consulta (SQL, PromQL, etc.)
Actualización y tiempo real	Actualización programada con Import; DirectQuery casi en vivo [4]	Auto-refresh de segundos a minutos; pensado para datos recientes [1]
Alertas	Alertas de datos sobre tarjetas y medidores en el servicio [3]	Reglas de alerta nativas con puntos de contacto [1]
Licenciamiento y costo	Desktop gratuito; compartir en el servicio exige licencia de pago [3]	OSS gratuito (AGPL); Cloud y Enterprise de pago [2]
Control de acceso	Áreas de trabajo, roles y seguridad a nivel de fila (RLS) con Microsoft Entra ID [3]	Roles por organización (Viewer, Editor, Admin) y permisos por carpeta; RBAC avanzado en Enterprise y Cloud [1]
Despliegue	Desktop para autoría y servicio SaaS de Microsoft	Autoalojado (Docker, instalador) o Cloud
Versionado	<code>.pbix</code> binario o proyecto <code>.pbip</code> [5]	JSON exportable o provisioning por archivos [1]

D3. Dashboard operativo, analítico y estratégico

El operativo sigue el estado actual para actuar en minutos (operador); el analítico explora tendencias y causas con filtros y periodos largos (analista); el estratégico resume pocos indicadores frente a metas para decidir (directivo) [6].

KPI	Tipo	Herramienta y motivo
Energía	Analítico	Power BI: se compara por día y periodo
Yield y Performance Ratio	Analítico	Power BI: medidas DAX sobre un modelo; el PR se normaliza con irradiación [7]
Ahorro (COP)	Estratégico	Power BI: usa tarifa parametrizable para el directivo
Disponibilidad y alarmas	Operativo	Grafana: alertas y datos recientes
% de datos válidos	Operativo y analítico	Grafana (panel <i>stat</i> con umbrales); Power BI para su tendencia

D4. Import, DirectQuery y actualización programada

Import copia los datos al modelo de Power BI y responde rápido; solo cambia cuando se actualiza. *DirectQuery* no guarda datos: envía una consulta a la fuente en cada interacción, así que depende de su velocidad [4]. La actualización programada (*scheduled refresh*) vuelve a importar en horas definidas, con un máximo diario según la licencia [8]. En SolarBI usaríamos *Import*: la tabla diaria es pequeña, el ETL corre una vez al día y el refresh puede lanzarse justo después. El auto-refresh de Grafana es distinto: el navegador repite las consultas contra la base cada pocos segundos o minutos y no almacena nada [1].

D5. Variables de Grafana frente a segmentadores de Power BI

Una variable de Grafana es un valor que se sustituye dentro del texto de la consulta antes de enviarla a la base [9]. Un segmentador de Power BI no reescribe consultas: agrega un filtro que se propaga por las relaciones del modelo y cambia el contexto de evaluación de las medidas DAX [3]. Para SolarBI, la variable `dispositivo` se define así y se usa en los paneles:

```
SELECT dispositivo_key AS __value, nombre AS __text FROM dwh.dim_dispositivo ORDER BY 1
```

```
SELECT ts AS time, p_ac_kw FROM silver.lectura_5min
WHERE $__timeFilter(ts) AND dispositivo_id IN ($dispositivo) ORDER BY ts
```

Al elegir el dispositivo 1, Grafana envía `dispositivo_id IN (1)`; con varios, `IN (1,2)`.

Bloque 2 · Gobernanza de datos

G1. Data governance, data management y data quality

Para DAMA-DMBOK, la gobernanza de datos es el ejercicio de autoridad y control, con decisiones compartidas, sobre la gestión de los activos de datos [10]. La *gestión de datos* (data management) es el conjunto de disciplinas que planifica y ejecuta esa gestión: arquitectura, almacenamiento, integración, seguridad. La *calidad de datos* es una de esas disciplinas y mide si el dato sirve para su uso [10]. En SolarBI: la gobernanza decide que una potencia válida está entre 0 y 5,5 kW; la gestión implementa el ETL; la calidad reporta el 94,12 % de datos válidos.

G2. Data owner, data steward y data custodian

El *data owner* responde por un dato ante la organización: decide su definición, uso y acceso. El *data steward* lo administra a diario: mantiene definiciones, vigila su calidad y resuelve dudas. El *data custodian* lo resguarda técnicamente: almacenamiento, respaldos, permisos y seguridad [10].

Propuesta para el equipo (es una asignación nuestra, no una norma): el *product owner* es *owner* de los datos y KPIs; el *BI analyst* y el *data modeler* son *stewards* (catálogo de KPIs y modelo estrella); el *data engineer* es *custodian* de PostgreSQL y del ETL.

G3. Contrato de datos

Un *data contract* es un acuerdo escrito entre quien produce y quien consume los datos. Debe incluir: esquema (columnas y tipos), unidades (kW, W/m², °C), frecuencia (5 minutos), zona horaria, responsable, niveles de calidad (rangos y % mínimo de válidos) y una regla para avisar los cambios. Si la fuente cambia un campo sin avisar, el tablero falla sin ruido. Ejemplo: si `p_ac_kw` pasara de kW a W, todas las lecturas superarían 5,5 y la regla de rango las rechazaría; el % de datos válidos caería a casi 0 y el yield saldría vacío.

G4. Fuente única de verdad y catálogo de KPIs

Una *single source of truth* es un único lugar autorizado donde vive cada dato o cálculo; los demás lo consumen sin reescribirlo. El *catálogo de KPIs* es el registro donde se define cada indicador para que todos lo entiendan igual [10].

Campo	Yield	% de datos válidos
Fórmula	energía kWh ÷ potencia instalada kWp	lecturas válidas ÷ lecturas leídas × 100
Unidad	kWh/kWp	%
Fuente	<code>dwh.fact_energia_dia</code> y <code>dwh.dim_dispositivo</code>	<code>dwh.fact_energia_dia</code>
Responsable	BI analyst (steward)	Data engineer (custodian)
Frecuencia	Diaria	Cada ejecución del ETL

Si un KPI se calcula en DAX y en SQL por separado, las fórmulas pueden divergir (por ejemplo, promedio de porcentajes frente a razón de sumas) y las dos herramientas mostrarían cifras distintas para el mismo indicador.

G5. Ley 1581 de 2012

La ley define dato personal como cualquier información vinculada o que pueda asociarse a una o varias personas naturales determinadas o determinables [11]. La telemetría de un inversor (potencia, irradiancia, temperatura) no describe a una persona, así que no es personal. Si el tablero mostrara qué operador atendió cada alarma, sí lo sería: identificaría a alguien y evaluaría su trabajo. Exigiría finalidad clara, autorización del titular, acceso restringido y seguridad [11]. Postura del equipo: usar solo datos del equipo, no recolectar datos de personas sin necesidad, y no subir credenciales al repositorio (`.env` excluido).

Bloque 3 · Automatización de procesos ETL

E1. ETL frente a ELT y capas Bronze, Silver y Gold

En ETL los datos se transforman antes de cargarse al destino; en ELT se cargan primero y se transforman dentro del destino aprovechando su capacidad [12]. En SolarBI: la **extracción** deja `telemetria.csv` en Bronze sin modificarlo; la **transformación** aplica las reglas de calidad y carga Silver (`silver.lectura_5min`); la agregación diaria llena Gold (`dwh.fact_energia_dia`). Como pandas transforma antes de cargar, el flujo es ETL. Power Query también extrae, transforma y carga al

modelo, así que sí es una herramienta ETL, pero limitada al modelo de Power BI: no orquesta ni gobierna capas externas.

E2. Carga completa, incremental e idempotencia

La carga completa reemplaza todo el destino en cada corrida; la incremental procesa solo lo nuevo o modificado [12]. Una carga es idempotente si ejecutarla una o varias veces deja el mismo resultado.

En `fact_energia_dia` se logra con la clave primaria (`fecha_key`, `dispositivo_key`) y un UPSERT [13]:

```
INSERT INTO dwh.fact_energia_dia (...) VALUES (...)  
ON CONFLICT (fecha_key, dispositivo_key) DO UPDATE SET energia_kwh = EXCLUDED.energia_kwh,  
...;
```

Si la fila existe se actualiza; si no, se inserta. Por eso, tras dos corridas, la tabla sigue con 3 filas.

E3. Tres formas de automatizar un flujo

	cron / Programador de tareas	Apache Airflow	Actualización programada de Power BI
Qué es	Programador del sistema operativo que lanza comandos a una hora [14]	Plataforma para definir, programar y monitorear flujos como DAGs de tareas en Python [15]	Función del servicio que vuelve a importar los datos al modelo [8]
Qué automatiza	Cualquier comando, por ejemplo <code>python etl/run_etl.py</code>	Un flujo completo con dependencias, reintentos y registro de ejecuciones	Solo la carga del modelo semántico, no el ETL
Cuándo conviene	Una tarea simple y periódica, como la de SolarBI	Varias tareas dependientes y necesidad de monitoreo	Mantener actualizados los informes ya publicados

E4. Actualización incremental en Power BI

Se crean dos parámetros de fecha reservados, `RangeStart` y `RangeEnd`, y se filtra la tabla con `fecha >= RangeStart and fecha < RangeEnd`. Luego se define una política: cuánto histórico conservar y cuánto reciente refrescar. Power BI divide la tabla en particiones y, en cada actualización, procesa solo las recientes [16]. Vale la pena con tablas grandes e históricos largos. Con `fact_energia_dia` (3 filas) no aporta; sí podría servir para `lectura_5min` cuando acumule meses de lecturas cada 5 minutos.

Bloque 4 · Internet de las Cosas (IoT)

I1. Arquitectura IoT por capas

Capas: **dispositivo** (inversor y sensores), **gateway o edge** (datalogger que lee los equipos y los reenvía), **red y broker** (transporte, por ejemplo MQTT), **almacenamiento** (archivos y PostgreSQL) y **aplicación** (Power BI y Grafana) [17]. En nuestra práctica el simulador reemplaza las tres primeras capas.

```

Inversor 5 kWp --> Gateway --> Broker MQTT --> [BRONZE] telemetria.csv (crudo)
| reglas de calidad
v
[SILVER] silver.lectura_5min --> Grafana
(serie cada 5 min)
| energía = p_ac_kw x 5/60
v
[GOLD] dwh.fact_energia_dia --> Power BI
(yield, ahorro)

```

12. MQTT

MQTT es un protocolo de mensajería *publish/subscribe*: los dispositivos publican en un *topic* y un *broker* reparte cada mensaje a quienes se suscribieron, sin que se conozcan entre sí [18]. Los topics son jerárquicos, separados por /. Hay tres niveles de QoS: 0, a lo sumo una vez; 1, al menos una vez (puede repetirse); 2, exactamente una vez [18]. Jerarquía propuesta:

```

pascualbravo/solar/<sitio>/<dispositivo>/<variable>
pascualbravo/solar/bloque-a/inversor-1/p_ac_kw
pascualbravo/solar/bloque-a/inversor-1/irradiancia_wm2

```

Para telemetría usaríamos QoS 1; los posibles duplicados ya los absorbe la regla de duplicados (también `dispositivo_id`).

13. Modbus, MQTT y OPC UA

	Modbus (RTU/TCP)	MQTT	OPC UA
Modelo de comunicación	Cliente/servidor (maestro/esclavo)	Publish/subscribe con broker	Cliente/servidor y publish/subscribe [19]
Uso típico	Leer registros de inversores y medidores [20]	Enviar telemetría del gateway a la nube [18]	Integración industrial con modelo de información [19]
Formato de los datos	Registros numéricos de 16 bits	Carga libre (JSON, binario)	Nodos tipados con nombre, unidad y tipo [19]
Seguridad	Sin seguridad nativa; se protege la red	TLS y usuario/contraseña [18]	Cifrado, firma y certificados integrados [19]

En inversores y medidores esperaríamos Modbus (RTU por RS-485 o TCP); MQTT, en el gateway hacia la nube.

14. Procesamiento por lotes y flujo continuo

En *batch* se procesan datos acumulados cada cierto tiempo; en *streaming*, cada dato al llegar [21]. Agregar la telemetría de 5 minutos a un día **gana** menos volumen, consultas más rápidas y KPIs estables como energía y yield. **Pierde** el detalle intradía: picos, nubosidad, fallas cortas y alarmas en tiempo real. Por eso Silver conserva los 5 minutos. Grafana atiende mejor la granularidad de 5 minutos y la vigilancia reciente; Power BI, la diaria y la analítica.

Bloque 5 · Control de versiones

V1. Git, GitHub y versionado de tableros

Git es un sistema de control de versiones distribuido que corre en el equipo; GitHub es una plataforma que aloja repositorios remotos y añade colaboración [22]. Un *repository* guarda los archivos y su historial; un *commit* es una instantánea con autor y mensaje; una *branch* es una línea paralela de trabajo; un *pull request* propone fusionar una rama y permite revisarla [22]. Un `.pbix` es un archivo binario comprimido: Git no muestra qué cambió ni puede fusionar versiones. El formato `.pbip` guarda el proyecto como carpeta de archivos de texto, con diferencias legibles [5]. Un dashboard de Grafana se versiona exportando su JSON, como `grafana/dashboard.json`.

Parte B · Práctica guiada: los mismos datos en dos herramientas

Se construyó un flujo completo sobre un inversor simulado de 5 kWp: Bronze (crudo), Silver (limpio) y Gold (resumen diario) en PostgreSQL, con dos vistas sobre la misma base. El código está en el repositorio ([etl/](#), [sql/](#), [powerbi/](#), [grafana/](#)).

Paso 1 · Bronze: datos crudos

`etl/simulador.py` genera 3 días de telemetría cada 5 minutos en `data/bronze/telemetry.csv` con anomalías inyectadas (potencia negativa, datos faltantes y filas duplicadas). Bronze no se modifica después de generarse.

```
PowerShell · solarbi-montoya-jimenez

> head -n 12 data/bronze/telemetry.csv
ts,dispositivo_id,p_ac_kw,irradiancia_wm2,temp_modulo_c
2026-10-05 00:00:00,1,0.0,0.0,22.0
2026-10-05 00:05:00,1,0.0,0.0,22.0
2026-10-05 00:10:00,1,0.0,0.0,22.0
2026-10-05 00:15:00,1,0.0,0.0,22.0
2026-10-05 00:20:00,1,0.0,0.0,22.0
2026-10-05 00:25:00,1,0.0,0.0,22.0
2026-10-05 00:30:00,1,0.0,0.0,22.0
2026-10-05 00:35:00,1,0.0,0.0,22.0
2026-10-05 00:40:00,1,0.0,0.0,22.0
2026-10-05 00:45:00,1,0.0,0.0,22.0
2026-10-05 00:50:00,1,0.0,0.0,22.0
```

Captura 1. Primeras filas de `data/bronze/telemetry.csv` (capa Bronze, sin modificar).

Paso 2 · Silver: reglas de calidad

Cada fila se rechaza por la primera regla que incumple, en este orden:

#	Regla	Criterio
1	Duplicados	Misma combinación <code>ts</code> + <code>dispositivo_id</code> ; se conserva la primera fila
2	Datos faltantes	Algún campo vacío
3	Rango físico	<code>p_ac_kw</code> entre 0 y 5,5 kW; <code>irradiancia_wm2</code> entre 0 y 1400 W/m ² ; <code>temp_modulo_c</code> entre -10 y 90 °C

Reporte de calidad de la ejecución (`data/silver/reporte_calidad.csv`):

Métrica	Valor
Filas leídas	884
Rechazadas por duplicado	20
Rechazadas por dato faltante	17
Rechazadas por fuera de rango	15
Filas válidas	832
% de datos válidos	94,12 %

```
PowerShell · solarbi-montoya-jimenez

> python etl/run_etl.py (ejecucion 1)
Reporte de calidad (Bronze -> Silver):
  filas_leidas           884
  rechazadas_duplicado   20
  rechazadas_faltante    17
  rechazadas_fuera_de_rango 15
  filas_validas          832
  pct_datos_validos      94.12
```

Captura 2. Reporte de calidad impreso por `python etl/run_etl.py`.

Paso 3 · Gold: carga en PostgreSQL

Un único comando, `python etl/run_etl.py`, lee Bronze, aplica las reglas, guarda Silver y carga `silver.lectura_5min`, `dwh.dim_dispositivo` y `dwh.fact_energia_dia` mediante `INSERT ... ON CONFLICT DO UPDATE`. La energía de cada intervalo es $p_ac_kw \times 5/60$ h. Se ejecutó dos veces y los conteos no cambiaron:

Tabla	Primera ejecución	Segunda ejecución
<code>silver.lectura_5min</code>	832	832
<code>dwh.dim_dispositivo</code>	1	1
<code>dwh.fact_energia_dia</code>	3	3

Resumen diario (Gold):

Fecha	Energía (kWh)	Lecturas leídas	Lecturas válidas	% válidos
2026-10-05	27,626	292	280	95,89
2026-10-06	26,393	294	278	94,56
2026-10-07	26,036	298	274	91,95

Expresión cron para lanzar el flujo todos los días a la medianoche:

```
0 0 * * * cd /ruta/al/repositorio && python etl/run_etl.py >> etl.log 2>&1
```

```
PowerShell · solarbi-montoya-jimenez

> python etl/run_etl.py (ejecucion 1)
Conteos en PostgreSQL despues de la carga:
silver.lectura_5min      832
dwh.dim_dispositivo      1
dwh.fact_energia_dia     3

> python etl/run_etl.py (ejecucion 2)
Conteos en PostgreSQL despues de la carga:
silver.lectura_5min      832
dwh.dim_dispositivo      1
dwh.fact_energia_dia     3
```

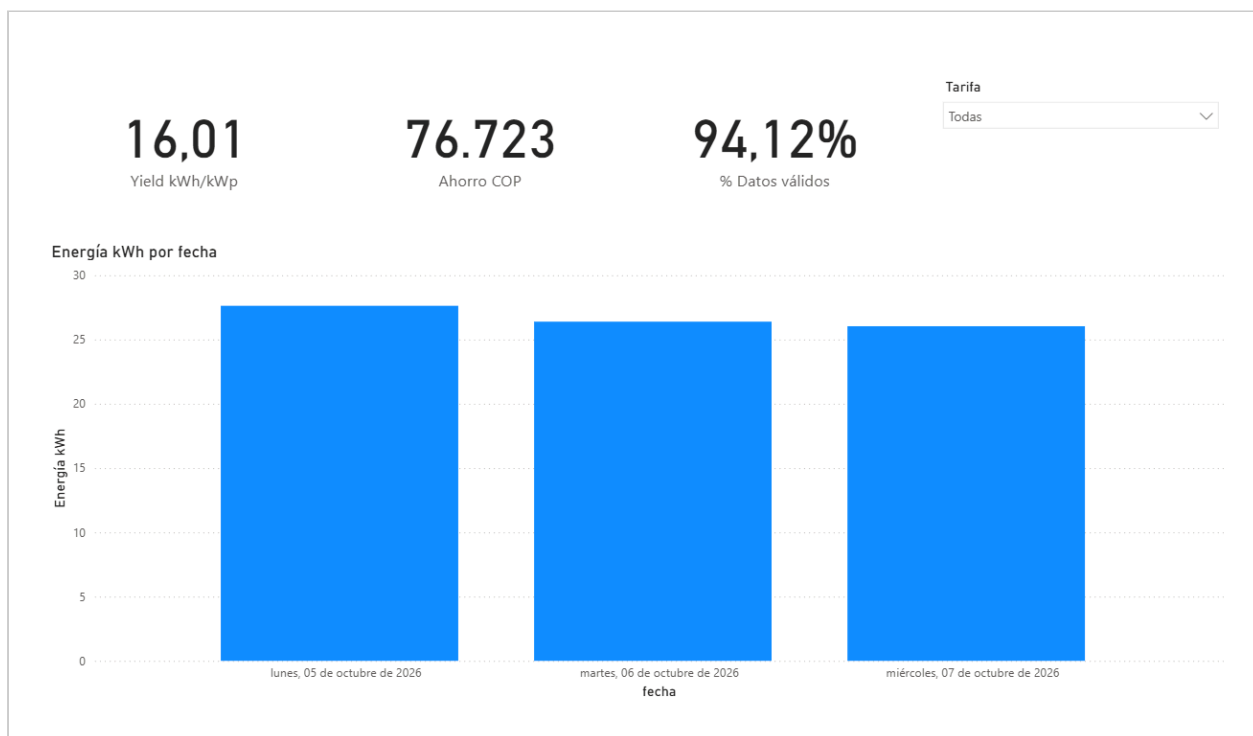
Captura 3. Conteos de PostgreSQL tras dos ejecuciones seguidas: no cambian (carga idempotente).

Paso 4 • Power BI

Power BI Desktop se conecta a PostgreSQL (localhost:5433, base solarbi) en modo Import. La página tiene una tarjeta de yield, un gráfico de columnas con la energía diaria y una tarjeta con el ahorro estimado. Las cifras salen de medidas DAX; el ahorro usa el parámetro Tarifa, con valor inicial de 958,38 COP/kWh, el costo unitario de EPM para estrato 4 en septiembre de 2026 [23].

```
Energía kWh = SUM ( fact_energia_dia[energia_kwh] )
Yield kWh/kWp = DIVIDE ( [Energía kWh], [Potencia instalada kWp] )
Ahorro COP = [Energía kWh] * [Tarifa Valor]
```

Valores de referencia con los datos actuales: energía 80,055 kWh, yield 16,01 kWh/kWp y ahorro de 76.723 COP.



Captura 4. Página de Power BI con yield (16,01 kWh/kWp), ahorro (76.723 COP con la tarifa por defecto de 958,38 COP/kWh), % de datos válidos, segmentador de tarifa y energía diaria.

Paso 5 • Grafana

Grafana OSS corre en Docker junto a PostgreSQL (`docker compose up -d`). El data source PostgreSQL y el dashboard se aprovisionan desde `grafana/`. El dashboard `SolarBI • Operación y calidad` tiene: una serie de tiempo con irradiancia y potencia cada 5 minutos que usa `$__timeFilter(ts)`, un panel con el porcentaje de datos válidos con umbrales de color, un panel de energía diaria y la variable `dispositivo`, aplicada en las tres consultas. El JSON exportado está en `grafana/dashboard.json`.

```
SELECT ts AS time, irradiancia_wm2 AS "Irradiancia (W/m²)", p_ac_kw AS "Potencia AC (kW)"
FROM silver.lectura_5min
WHERE $__timeFilter(ts) AND dispositivo_id IN ($dispositivo)
ORDER BY ts
```



Captura 5. Dashboard `SolarBI • Operación y calidad` en Grafana OSS: serie de tiempo de irradiancia y potencia, % de datos válidos (94,12 %), energía diaria y variable `Dispositivo`.

Paso 6 • Reflexión

Con los mismos datos, Grafana resolvió mejor la vista operativa: una consulta SQL con `$__timeFilter` dibujó la curva de irradiancia y potencia cada 5 minutos, y la variable de dispositivo cambió las tres consultas sin tocar nada más. Power BI resolvió mejor lo analítico: la relación entre hecho y dimensión y las medidas DAX de yield y ahorro, que se recalculan al cambiar la tarifa; en Grafana esa lógica habría que repetirla en cada consulta. En Grafana nos costó más el ambiente (Docker, el acceso de administrador y fijar el rango de fechas, porque los datos son de solo tres días). En Power BI nos costó la presentación (eje de fechas continuo, ahorro abreviado) y encontrar una tarifa con fuente citable. Para el operador de la planta usaríamos Grafana, por la inmediatez y las alertas; para un directivo, Power BI, por indicadores como yield y ahorro con filtros.

Parte C · Repositorio en GitHub

Repositorio: <https://github.com/danielita2508/solarbi-montoya-jimenez>

El repositorio es público y contiene el README con los pasos para reproducir la práctica, el `.gitignore`, el flujo ETL, el SQL, los datos Bronze y Silver, las medidas de Power BI y el dashboard de Grafana. Las credenciales viven en un archivo `.env` excluido del repositorio; solo se versiona `.env.example`.

Integrante	Usuario de GitHub
Daniela Montoya Quintero	@danielita2508
Isaac Madera Jiménez	@IsaVonxz-type

[Captura 6: Settings → Collaborators con los dos integrantes]



PlatformSolutionsResourcesOpen SourceEnterprisePricing

Q

Sign in

Sign up

🔍

danielita2508 / solarbi-montoya-jimenez

<> CodeIssuesPull requestsActionsProjectsSecurity and qualityInsights

Commits

main

All users

All time

Commits on Oct 7, 2026

Completar la tabla de quién hizo qué en el README

IsaVonxz-type committed 5 minutes ago

30664a9

<>

Agregar el PDF de la consulta con las partes A, B y C

IsaVonxz-type committed 5 minutes ago

ce76db5

<>

Escribir la reflexión comparativa entre Power BI y Grafana

IsaVonxz-type committed 5 minutes ago

1ce5288

<>

Agregar capturas de evidencia de los pasos 1 a 5 de la Parte B

IsaVonxz-type committed 5 minutes ago

dc65998

<>

Redactar las respuestas de la Parte A con sus referencias

IsaVonxz-type committed 5 minutes ago

070c9e9

<>

Documentar la tarifa de EPM como valor inicial del ahorro y el uso del proyecto .pbip

IsaVonxz-type committed 5 minutes ago

47b274e

<>

Agregar proyecto de Power BI (.pbip) con modelo, medidas DAX y página de resumen

IsaVonxz-type committed 5 minutes ago

2399b19

<>

Commits on Oct 6, 2026

Completar tabla de integrantes con nombres y usuarios de GitHub

IsaVonxz-type committed 1 hour ago

89843a5

<>

Commits on Oct 5, 2026

Escribir README con pasos para reproducir la práctica, reglas de calidad y expresión cron

danielita2508 and claude committed yesterday

f8cfa8

<>

Documentar conexión a PostgreSQL y medidas DAX de yield, energía y ahorro para Power BI

danielita2508 and claude committed yesterday

6e450e5

<>

Agregar dashboard de Grafana con series de tiempo, porcentaje de datos válidos y variable de dispositivo

danielita2508 and claude committed yesterday

185b9b8

<>

Agregar docker-compose con PostgreSQL y Grafana OSS para el ambiente local

danielita2508 and claude committed yesterday

3da397a

<>

Implementar ETL con reglas de calidad y carga idempotente a PostgreSQL mediante UPSERT

danielita2508 and claude committed yesterday

c9f8495

<>

Definir esquemas silver y dwh con claves primarias para permitir cargas idempotentes

danielita2508 and claude committed yesterday

e5d0ac3

<>

Agregar simulador de telemetría que genera la capa Bronze con anomalías inyectadas

danielita2508 and claude committed yesterday

9aed722

<>

Agregar estructura inicial del repositorio, .gitignore y plantilla de variables de entorno

2dd7773

<>

Captura 7. Historial de commits de la rama `main` en GitHub, con commits de los dos integrantes (`danielita2508` e `IsaVonxz-type`).

13

Referencias

1. Grafana Labs. *Grafana documentation* (paneles, variables, alertas, fuentes de datos). <https://grafana.com/docs/grafana/latest/>
2. Grafana Labs. *Grafana Cloud y Grafana OSS*. <https://grafana.com/products/cloud/> y <https://grafana.com/oss/grafana/>
3. Microsoft. *Power BI documentation*. <https://learn.microsoft.com/power-bi/>
4. Microsoft. *Dataset modes in the Power BI service* (Import y DirectQuery). <https://learn.microsoft.com/power-bi/connect-data/service-dataset-modes-understand>
5. Microsoft. *Power BI Desktop projects (PBIP)*. <https://learn.microsoft.com/power-bi/developer/projects/projects-overview>
6. Few, S. (2013). *Information Dashboard Design* (2.^a ed.). Analytics Press.
7. IEC 61724-1. *Photovoltaic system performance – Monitoring*. <https://webstore.iec.ch/publication/33622>
8. Microsoft. *Data refresh in Power BI*. <https://learn.microsoft.com/power-bi/connect-data/refresh-data>
9. Grafana Labs. *Variables*. <https://grafana.com/docs/grafana/latest/dashboards/variables/>
10. DAMA International. *DAMA-DMBOK: Data Management Body of Knowledge* (2.^a ed.). <https://www.dama.org/cpages/body-of-knowledge>
11. Congreso de Colombia. *Ley 1581 de 2012*, art. 3. <https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=49981>
12. Databricks. *Medallion architecture*. <https://www.databricks.com/glossary/medallion-architecture>
13. PostgreSQL. *INSERT ... ON CONFLICT*. <https://www.postgresql.org/docs/current/sql-insert.html>
14. Microsoft. *Task Scheduler*. <https://learn.microsoft.com/windows/win32/taskschd/task-scheduler-start-page>
15. Apache Airflow. *Documentation*. <https://airflow.apache.org/docs/>
16. Microsoft. *Incremental refresh and real-time data for semantic models*. <https://learn.microsoft.com/power-bi/connect-data/incremental-refresh-overview>
17. Microsoft. *Azure IoT reference architecture*. <https://learn.microsoft.com/azure/architecture/reference-architectures/iot>
18. OASIS. *MQTT Version 5.0*. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
19. OPC Foundation. *OPC Unified Architecture*. <https://opcfoundation.org/about/opc-technologies/opc-ua/>
20. Modbus Organization. *Modbus specifications*. <https://modbus.org/specs.php>
21. Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly.
22. Chacon, S. y Straub, B. *Pro Git*. <https://git-scm.com/book>
23. Empresas Públicas de Medellín E.S.P. (2026, 16 de septiembre). *Tarifas y Costo de Energía Eléctrica – Mercado Regulado, septiembre de 2026* [PDF]. https://www.epm.com.co/content/dam/epm/clientes-y-usuarios/energia/tarifas-energia/Tarifas%202026/9.PublicacionTarifasSeptiembre162026_ANT_OM.pdf (consultado el 6 de octubre de 2026). Tarifa usada en la Parte B (Power BI, medida Ahorro COP).